Планировщик отпусков

КОРПОРАТИВНЫЙ ГРАФИК ОТПУСКОВ И СОГЛАСОВАНИЙ

Стенды, CI/CD и выпуск

Окружения, конвейеры, версии и эксплуатация · версия 1.0.0

30 октября 2026

© Alexander Yurasov

Содержание

1 О документе 3

2 Окружения 4

2.1 Локальное окружение 4

3 Конвейеры непрерывной интеграции 6

4 Ветки, версии и выпуск 9

5 Миграции базы данных 10

6 Резервное копирование и мониторинг 11

7 Доступы 12

1 О документе

Документ описывает окружения «Планировщика отпусков», конвейеры непрерывной интеграции в GitHub и Gitea, порядок выпуска версий, применение миграций, резервное копирование и мониторинг. Пошаговые инструкции для администратора стенда — «Руководство по развёртыванию».

ИСТОЧНИК	ЧТО ЗАДАЁТ
`docker-compose.yml`	Контейнеры стенда: postgres, server, web, backup; профиль `monitoring` — Prometheus
`docker-compose.replica.yml`	Дополнение: реплика PostgreSQL, архив WAL, физические копии
`deploy/`	Конфигурация Prometheus, оповещения, панель Grafana; скрипты реплики и физических копий
`.github/workflows/ci.yml`	Конвейер GitHub Actions
`.gitea/workflows/server-ci.yml`	Конвейер Gitea Actions с развёртыванием стенда
`Makefile`	Типовые команды разработчика
`docs/versions.md`, `CHANGELOG.md`	История версий, авторство, технический журнал

2 Окружения

ОКРУЖЕНИЕ	НАЗНАЧЕНИЕ	ДАННЫЕ	КТО РАБОТАЕТ
Локальное	Разработка и отладка	Демо-данные (`make seed`)	Разработчик
CI (GitHub, Gitea)	Проверка каждого push и PR	Тестовая БД `vacation_test` , демо-данные для e2e интерфейса	Автоматика
Стенд Gitea	Демонстрация и приёмка версии	Том `db-data` стенда	Команда, заказчик
Рабочий контур	Работа организации	Рабочая БД, резервные копии	Сотрудники

2.1 Локальное окружение

КОМАНДА	ДЕЙСТВИЕ
`make up`	PostgreSQL в Docker
`make seed`	Миграции и демо-данные (131 сотрудник, 14 отделов)
`make server-dev` / `make web-dev`	API на `:8080` , SPA на `:5173` с прокси `/api`
`make test`	Тесты сервера на БД `vacation_test`
`cd web && npm test` / `npm run e2e`	Модульные тесты клиента / e2e интерфейса (Playwright)
`make lint`	ESLint сервера и клиента
`make sbom`	SBOM CycloneDX и отчёт лицензий
`make docs-diagrams` / `make docs`	Схемы в PNG / комплект документов в PDF и DOCX
`make compose-full`	Полный стенд в контейнерах на `:80`
`npm run db:seed:load -- --employees 2000 --history 5` (в `server/`)	Нагрузочная БД заданной численности с историей (имя БД должно содержать `load`); по умолчанию 5000 сотрудников

КОМАНДА	ДЕЙСТВИЕ
`node tools/load/run.mjs --profile average --population 2000`	Нагрузочный прогон по модели рабочего дня: роли, вкладки с потоком событий, профили `smoke`, `average`, `spike`, `stress`, `soak` (`docs/performance/`)
`tools/load/pinned.sh`	Прогон сайзинга: API и PostgreSQL на заданных ядрах, сбор доказательств
`scripts/build-dist.sh`	Дистрибутив версии: исходные тексты, образы, комплект установки, SHA-256

Секреты — только в `.env` (образец `.env.example`): `DATABASE_URL`, `JWT_SECRET` не короче 32 символов, с v0.4.0 — `SMTP_PASSWORD` и `OIDC_CLIENT_SECRET`, с v0.5.0 — `REPLICATION_PASSWORD` (стенд с репликой) и необязательный `METRICS_TOKEN`. В git их нет. С v0.6.0 — необязательный `JOBS_POLL_SECONDS` (60): как часто процесс проходит плановые задачи — извещения, эскалацию, напоминания, очистку журнала входов; `0` выключает таймер (тесты вызывают задачи сами). Почта и единый вход необязательны: без `SMTP_HOST` и `OIDC_ISSUER` стенд работает с уведомлениями в интерфейсе и входом по паролю.

3 Конвейеры непрерывной интеграции

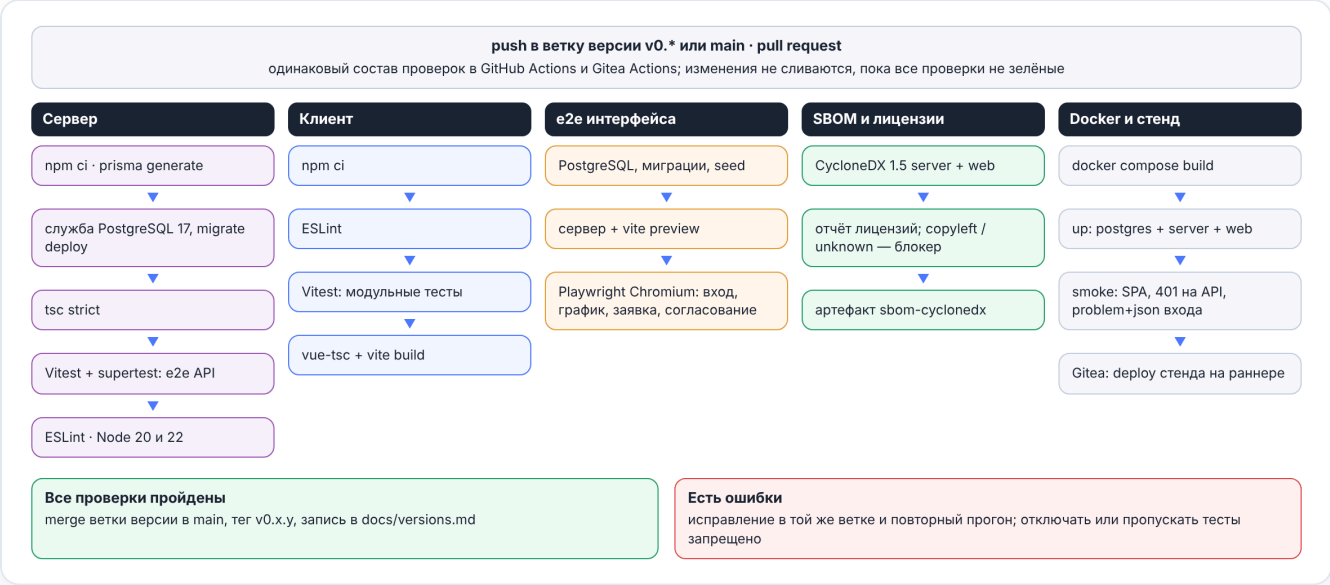


Рис. 1. Состав проверок CI

Оба конвейера запускаются на push в `main` и ветки версий `v*` (GitHub — также на pull request) и выполняют одинаковый набор проверок:

ЗАДАНИЕ	ШАГИ	ЧТО ЛОВИТ
Сервер	`npm ci`, `prisma generate`, миграции на чистой PostgreSQL 17, обновление с прошлого выпуска на данных (`scripts/upgrade-check.sh`: миграции `main` в отдельной схеме, данные прошлого выпуска, миграции ветки, сверка схемы), `tsc` strict, Vitest (модульные и e2e API), ESLint; Node 20 и 22	Ошибки логики, прав, транзакций, расхождение контракта и маршрутов, гонки, миграции, падающие только на данных
Клиент	`npm ci`, ESLint, Vitest модулей `lib/*`, `vue-tsc` + сборка Vite	Ошибки типов и вычислений дат, графика, статусов

ЗАДАНИЕ	ШАГИ	ЧТО ЛОВИТ
e2e интерфейса	PostgreSQL, миграции, seed, сборки, Chromium; Playwright: вход, график, заявка с подбором дат и согласованием, команда, график работы, кадровая служба и импорт, одноразовый пароль, замещение, извещение и ознакомление, кампания планирования и утверждение графика с формой Т-7, CSP всех экранов	Регрессии экранов и сценариев, нарушения CSP
SBOM	CycloneDX 1.5 для сервера и клиента, `scripts/license-report.mjs`	Copyleft и неизвестные лицензии — блокер
Docker (GitHub)	Сборка образов и дымовой прогон полного стенда: SPA, вход через nginx, `/metrics` внутри стенда и его отсутствие снаружи, поток событий через nginx (демо-данные, вход, билет, первое событие)	Ошибки Dockerfile, nginx, старта с миграциями; буферизация SSE; утечка метрик наружу
Реплика (GitHub)	Стенд с `docker-compose.replica.yml`: реплика в потоке, изменения видны на ней, запись на неё отклоняется, архив WAL и физическая копия, метрики реплики; авария основного (`kill`), повышение реплики, `server` на реплике — вход, чтение и запись через nginx, архив новой линии времени	Поломки репликации, архива и процедуры переключения
Нагрузка (GitHub)	300 сотрудников с историей за год, сборка сервера, профиль `smoke` с `--slo`	p95 чтения > 300 мс, записи > 500 мс, ошибки и сбои потока событий
Документы (GitHub)	`python3 tools/docs/gen_tables.py --check`	Каталог операций, словарь данных, коды ошибок и показатели кода в документах разошлись с кодом

ЗАДАНИЕ	ШАГИ	ЧТО ЛОВИТ
Deploy (Gitea)	После всех проверок: `docker compose -p vacation-planner up -d --build` на раннере `deploy` и дымовые пробы, включая `/metrics`	Развёртывание стенда версии

Правила:

- красный конвейер блокирует слияние; отключать, пропускать или ослаблять тесты запрещено — исправляется причина;
- «нестабильный тест» не причина: в v0.3.0 так нашлась ошибка дымовой пробы (`CODE=$(curl …)` под `bash -e` падал при обрыве соединения до готовности API);
- дифф контракта, SBOM и документов — в том же PR, что и код.

4 Ветки, версии и выпуск



Рис. 2. Жизненный цикл выпуска

1. Номер версии `vX.Y.Z`. До выпуска 1.0.0 — `v0.x.y` (`x` — крупные накопленные изменения, `y` — исправления); с 1.0.0: `X` растёт при несовместимых изменениях API или данных, `Y` — при новых возможностях, `Z` — при исправлениях.
2. Работа над версией — в ветке `vX.Y.Z`; отдельные истории — в ветках `story/<id>` от ветки версии с pull request по шаблону.
3. `main` получает только принятые версии: merge ветки версии и тег `vX.Y.Z` на merge-коммите. Merge и тег ставит владелец.
4. Перед выпуском обновляются `docs/versions.md` (что получил пользователь, авторство), `CHANGELOG.md` (технически), заметки к выпуску `docs/release-notes/`, версии в `package.json`, документы (`make docs`) и SBOM; для поставки собирается дистрибутив `scripts/build-dist.sh` с контрольными суммами.
5. Версия с новыми возможностями (`vX.Y.0`) выпускается только после полного прогона всех тестов в обоих конвейерах; с 1.0.0 — и после нагрузочного прогона профилей `average` и `spike` (`docs/performance/runbook.md`).

Порядок выпуска: зелёные конвейеры на ветке версии → обновлены история версий, журнал, документы, SBOM → merge в `main` с тегом → развёртывание стенда Gitea и дымовые пробы → приёмка → обновление рабочего контура по регламенту `docs/runbooks/update.md` (резервная копия → новые образы → `docker compose up -d` → проверка; откат — там же).

5 Миграции базы данных

- Схема — `server/prisma/schema.prisma`; миграции применяются автоматически при старте контейнера `server` (`prisma migrate deploy`).
- Перед обновлением рабочего контура снимается внеочередная резервная копия.
- Опасные изменения (удаление, переименование) выполняются в два выпуска через добавляющие шаги.
- Seed идемпотентен и ничего не удаляет; на рабочем контуре его запуск не требуется.
- Каждая ветка версии проверяется обновлением с данных `main` (шаг CI «Обновление с прошлого выпуска на данных»): миграция, которая проходит на пустой базе и падает на данных, не попадёт в выпуск. В v0.6.0 так найдена миграция замещения, падавшая с «pending trigger events» — порядок операций исправлен до выпуска.

6 Резервное копирование и мониторинг

ЗАДАЧА	РЕАЛИЗАЦИЯ
Резервное копирование	Сервис `backup`: `pg_dump -Fc` раз в `BACKUP_INTERVAL` секунд (сутки), хранение `BACKUP_KEEP_DAYS` (14) дней, том `db-backups`; запись атомарная (через `.tmp`)
Проверка восстановления	Раз в месяц — `pg_restore` последней копии в пустую БД и сверка числа сотрудников и заявок (руководство по развёртыванию)
Пробы	`/healthz` — процесс жив; `/readyz` — доступна БД; healthcheck PostgreSQL в compose
Журналы	stdout контейнеров; `docker compose logs`; ротация средствами Docker
Метрики и оповещения	`/metrics` сервера; Prometheus — профиль `monitoring`, правила `deploy/monitoring/alerts.yml`; панель `deploy/monitoring/grafana-dashboard.json` импортируется в Grafana организации
Реплика и архив WAL	`docker-compose.replica.yml`: горячий резерв, архив WAL (`archive_timeout` 5 минут), ежедневные физические копии с хранением 7 штук. RPO — секунды (реплика) или до 5 минут (архив), RTO — 15 минут
Проверка восстановления на момент времени	Раз в квартал — физическая копия и архив WAL в пустой каталог, `recovery.signal`, сверка данных (руководство по развёртыванию). При подготовке v0.5.0 проверено на PostgreSQL: данные восстановлены до последнего заархивированного сегмента, транзакция из незаархивированного сегмента ожидаемо потеряна

7 Доступы

РОЛЬ	ДОСТУП
Разработчик	Репозитории GitHub и Gitea; права на push в ветки историй
Владелец (ревью)	CODEOWNERS: `contracts/`, `docs/specs/`, миграции, `auth/`, workflows, зависимости
Оператор стенда	SSH к серверу, `.env` стенда вне git, раннер `deploy`
Администратор приложения	Учётная запись с признаком администратора внутри приложения